SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.	3
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.	3
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.	3
5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.	3
6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.	3

7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.	3
8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.	3
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.	3
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.	3
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.	3
13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.	3
14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.	3
15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.	3
16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.	3
17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.	3

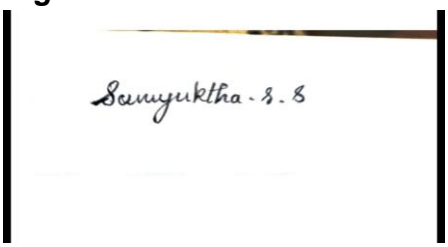
18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.	3
19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.	3
20	A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.	3
21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.	3
22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.	3
23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.	3
24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.	3
25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.	3
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.	3
27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.	3
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.	3

29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.	3
30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.	3

Code of Conduct:

I Samyuktha S S(192311056) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Q1. Testing as an Engineering Activity

Answer:

Testing should be treated as a **systematic engineering activity** rather than random execution of test cases. Random testing may produce inconsistent results and can miss important defects.

1. Structured Testing Approach

The testing team should first understand the requirements and prepare a proper test plan. Test cases should then be designed based on the expected behaviour of the software.

Suitable techniques include:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Table Testing
- State Transition Testing
- Regression Testing

2. Benefits of Structured Test Design

Structured test design helps to:

- Improve test coverage
- Detect defects systematically
- Reduce missed test scenarios
- Make testing repeatable
- Identify high-risk areas
- Improve overall software quality

3. Example

For a banking login system, instead of randomly entering usernames and passwords, testers can prepare cases for:

- Correct username and password
- Incorrect username
- Incorrect password
- Empty fields
- Multiple failed login attempts

Each test has a defined expected result.

Conclusion

Treating testing as an engineering activity makes testing **planned, systematic, measurable, and repeatable**. Structured test design improves defect detection and produces more reliable software.

Q2. Importance of a Well-Defined Testing Process

Answer:

A well-defined testing process provides a standard approach for planning, designing, executing, and reporting tests. It is especially important for real-time systems such as banking applications, where failures can cause financial loss.

1. Defined Testing Stages

A standard process includes:

1. Requirement analysis
2. Test planning
3. Test-case design
4. Test execution
5. Defect reporting
6. Retesting
7. Regression testing
8. Test closure

2. Benefits

A defined process provides:

- Consistent testing
- Better test coverage
- Early defect detection
- Easier defect tracking
- Reduced testing errors
- Improved software reliability

3. Example

For an ATM withdrawal system, testers can systematically test:

- Valid PIN
- Invalid PIN
- Sufficient balance
- Insufficient balance
- Withdrawal limit

- Invalid amount

This is more reliable than randomly testing a few transactions.

Conclusion

A well-defined testing process ensures that important functions are tested consistently. In banking systems, this improves reliability, security, and customer confidence.

Q3. Testing as a Structured SDLC Process

Answer:

Testing should be performed throughout the **Software Development Life Cycle (SDLC)** rather than only after coding is completed. In an e-commerce system, inconsistent testing can lead to integration and system-level defects.

1. Testing Across SDLC

Requirement Phase:

Testers review requirements and identify unclear or missing conditions.

Design Phase:

Testers review the design and identify possible integration and usability risks.

Development Phase:

Developers perform unit testing, while testers prepare test cases.

Integration Testing:

Different modules such as login, product search, cart, and payment are tested together.

System Testing:

The complete application is tested against its requirements.

Regression Testing:

Existing functionality is tested after changes or defect fixes.

2. Importance of Process Discipline

Following a standard process:

- Reduces missed defects
- Improves communication
- Provides better test coverage
- Makes testing consistent
- Detects defects earlier

Example

In an e-commerce application, the cart may work independently but fail when connected to the payment module. Integration testing helps identify this type of defect.

Conclusion

Testing throughout the SDLC ensures that defects are identified at the appropriate stage. Process discipline improves software quality and reduces the risk of major defects reaching the final system.

Q4. Error, Defect, and Failure

Answer:

An **error, defect, and failure** are related but different concepts in software testing.

1. Error

An **error** is a human mistake made during requirements analysis, design, coding, or another development activity.

Example:

A developer misunderstands the login requirement.

2. Defect

A **defect** is a flaw in the software caused by an error.

Example:

The developer writes code that accepts an incorrect password.

3. Failure

A **failure** occurs when the software executes the defective code and produces an incorrect result.

Example:

A user enters an incorrect password, but the system still allows login.

Relationship

The relationship can be represented as:

Human Error → Software Defect → Execution → Failure

Practical Example

If a developer incorrectly implements a password-validation condition, the incorrect code is the **defect**. When a user enters an invalid password and is incorrectly allowed to log in, it becomes a **failure**.

Conclusion

Understanding the difference between error, defect, and failure helps testers report problems accurately and allows developers to identify their causes more effectively.

Q5. Early Defect Detection

Answer:

The principle of **early testing** can be applied to detect defects as early as possible in the software development process. Finding defects early reduces the cost and effort required to fix them later.

1. Apply Early Testing

Testers should be involved from the requirement and design stages rather than waiting until the software is completely developed.

They can:

- Review requirements
- Identify ambiguities
- Review designs
- Prepare test cases early
- Identify high-risk areas

2. Why Early Detection Is Important

A defect found during the requirement stage is usually much cheaper to fix than the same defect discovered after deployment.

Early testing helps to:

- Reduce development cost
- Reduce rework
- Improve requirements
- Prevent defect propagation
- Improve software quality

3. Example

Suppose a banking requirement incorrectly states the maximum transaction limit.

If testers identify the ambiguity during requirement review, it can be corrected before coding begins.

If the same problem is discovered after deployment, it may require code changes, database changes, retesting, and possibly customer support.

Conclusion

Early testing improves testing effectiveness by identifying defects before they spread to later stages. It reduces cost, effort, and the possibility of serious production failures.

Q6. Tester Involvement During Requirement Analysis

Answer:

Testers should participate during the **requirement analysis phase** because unclear requirements can later become implementation defects. Early tester involvement helps ensure that requirements are clear, complete, consistent, and testable.

1. Role of the Tester

A tester can:

- Review requirements
- Identify ambiguous statements
- Find missing conditions
- Ask clarification questions
- Identify possible edge cases
- Check whether requirements are testable

2. Healthcare Example

Suppose a healthcare requirement states:

"The system should quickly display patient information."

The word **"quickly"** is ambiguous.

A tester can ask for a measurable requirement, such as:

"Patient information should be displayed within 2 seconds under normal conditions."

Now the requirement can be properly tested.

3. Benefits of Early Involvement

Early involvement:

- Detects defects before coding
- Reduces rework
- Improves requirement quality
- Improves test planning
- Reduces development cost

Conclusion

Testers should be involved from the requirement phase because testing is not only a final-stage activity. Early involvement helps remove ambiguity and prevents requirements-related defects from entering later stages.

Q7. Origin of Defects in the Software Lifecycle

Answer:

Defects can originate at different stages of the software development lifecycle. In the given payroll system, unclear and ambiguous requirements can lead to incorrect implementation and incorrect outputs.

1. Requirement Stage

If requirements are unclear, developers may interpret them differently.

For example, a payroll requirement may not clearly specify how overtime should be calculated.

2. Design Stage

Incorrect interpretation of requirements can result in an incorrect system design.

3. Coding Stage

Developers may implement incorrect formulas or logic based on the unclear requirements.

4. Testing Stage

If test cases are also based on unclear requirements, the defect may remain undetected.

5. Production Stage

The defect finally becomes visible when the system produces incorrect salary calculations.

Prevention Through Requirement Clarity

Requirements should be:

- Clear
- Complete
- Consistent
- Unambiguous
- Testable
- Measurable

Example

If the overtime rate is ₹200 per hour, the requirement should clearly specify the calculation method. This prevents different developers from interpreting the rule differently.

Conclusion

Many software defects originate from unclear requirements. Proper requirement reviews, tester involvement, clarification meetings, and measurable requirements can prevent such defects from progressing into design and implementation.

Q8. Boundary Value Analysis

Answer:

If a system crashes when boundary input values are entered, it indicates that **boundary conditions were not adequately tested**. The defect can be classified as a **functional or validation defect**, depending on the exact requirement and behaviour.

1. Boundary Value Analysis

Boundary Value Analysis tests values at and around the limits of an input range.

For example, suppose a system accepts values from **1 to 100**.

Important test values are:

- 0 — below minimum
- 1 — minimum
- 2 — just above minimum
- 99 — just below maximum
- 100 — maximum
- 101 — above maximum

2. Why the Defect Occurs

If testers only test normal values such as 50, they may miss errors occurring at 1, 100, 0, or 101.

3. Practical Example

Suppose an ATM allows withdrawal amounts up to ₹50,000.

The tester should test:

- ₹49,999
- ₹50,000
- ₹50,001
- ₹0
- Negative amounts

The system should accept valid values and reject invalid values without crashing.

Conclusion

Boundary Value Analysis improves test coverage by focusing on values near input limits. Testing boundary and just-outside-boundary values could have detected the issue before deployment.

Q9. Defect Repository for Better Test Case Design

Answer:

A **defect repository** stores information about defects identified during software development and testing. Historical defect information can be used to improve future test cases and prevent repeated defects.

1. Analyze Historical Defects

The testing team should review:

- Frequently affected modules
- Repeated defect types
- Defect severity
- Root causes
- Defects from previous releases

2. Use Defects to Design Test Cases

If previous releases contain repeated payment defects, testers can create additional test cases for:

- Invalid payment details
- Duplicate transactions
- Failed payments
- Network interruptions
- Incorrect payment amounts

These tests can be included in regression testing.

3. Benefits

A defect repository helps to:

- Improve test coverage
- Identify high-risk areas
- Prevent recurring defects
- Improve regression testing
- Support root-cause analysis
- Increase software quality

Example

If a shopping application repeatedly calculates discounts incorrectly, previous defects can be used to create test cases for minimum purchase amounts, maximum discounts, and different combinations of offers.

Conclusion

A defect repository should not only record defects. It should also provide useful historical information for improving test design and preventing similar defects in future releases.

Q10. Functional Defect in an ATM Withdrawal System

Answer:

A **functional defect** occurs when a software function does not behave according to its specified requirements. In the given ATM scenario, incorrect transaction logic causes the withdrawal operation to fail.

1. Identify the Functional Defect

Suppose a customer has sufficient balance and enters a valid withdrawal amount, but the ATM:

- Rejects the valid transaction, or
- Deducts an incorrect amount, or
- Dispenses incorrect cash.

This indicates that the withdrawal function is not working according to its requirements.

2. Example

Suppose the customer requests ₹5,000 and has sufficient balance.

Expected Result:

₹5,000 should be dispensed and ₹5,000 should be deducted from the account.

Actual Result:

The system deducts ₹5,000 but dispenses only ₹4,000.

This is a functional defect in the transaction-processing logic.

3. How It Could Have Been Prevented

The defect could have been detected through:

- Functional testing
- Unit testing
- Integration testing
- Boundary Value Analysis
- Decision Table Testing
- Regression testing

Test cases should cover valid withdrawals, insufficient balance, withdrawal limits, invalid amounts, and transaction failures.

Conclusion

The ATM problem is a functional defect because the withdrawal operation does not produce the required result. Systematic functional and integration testing with suitable test cases could have detected the problem before deployment.

Q11. Defect Clustering and Testing Efficiency

Answer:

1. Meaning of Defect Clustering

Defect clustering is a software testing principle that states that **a small number of modules usually contain a large number of defects**.

Testers can use historical defect data to identify modules where defects occur frequently.

2. Application in Testing

The testing team can:

- Analyze previous defect reports.
- Identify modules with repeated or high-severity defects.
- Give these modules higher testing priority.
- Perform additional functional and regression testing.

3. Example

In an e-commerce application, suppose most defects are repeatedly found in the **payment and checkout modules**. Testers can spend more testing effort on these modules instead of giving equal effort to every module.

4. Benefit

Defect clustering improves testing efficiency because limited testing time and resources are focused on **high-risk areas**.

Conclusion:

Using defect clustering helps testers prioritize important modules, detect more defects, and improve overall software quality.

Q12. Classification of Performance Defects

Answer:

1. Identify the Defect

If a web application becomes very slow when many users access it simultaneously, it is a **performance defect**.

It may occur because the system cannot handle the required workload efficiently.

2. Possible Causes

- High CPU or memory usage
- Inefficient database queries
- Poorly optimized code
- Insufficient server resources

- Improper handling of concurrent users

3. Testing Approach

Testers can perform:

- **Load testing** – checks performance under expected user load.
- **Stress testing** – checks behavior beyond normal capacity.
- **Performance testing** – measures response time and resource usage.

4. Importance of Classification

Correct classification helps developers understand the nature of the problem and select the appropriate solution.

Example:

If a shopping website takes 10 seconds to display the checkout page during peak usage, the issue can be classified as a performance defect and investigated through load testing.

Conclusion:

Classifying performance defects helps the team identify the correct testing method and improve system response time and reliability.

Q13. Using Historical Defect Data for Regression Testing

Answer:

1. Meaning

Historical defect data contains information about defects that were previously found, their locations, causes, severity, and the test cases that detected them.

This information can be reused during regression testing.

2. Application

Testers can:

- Identify previously defective modules.
- Re-run test cases related to old defects.
- Include frequently failing scenarios in regression suites.
- Give higher priority to high-risk areas.

3. Example

Suppose a banking application previously had a defect where money was deducted twice during a transaction. After a software update, the tester should include this scenario in regression testing.

4. Benefits

Using historical data:

- Improves test coverage.
- Reduces repeated defects.
- Saves test design time.
- Helps focus on high-risk areas.

Conclusion:

Historical defect data makes regression testing more effective by ensuring that previously problematic areas are tested again after software changes.

Q14. Tester Involvement in Integration Testing

Answer:

1. Problem

Integration defects can occur when different software modules do not communicate correctly. Poor communication between developers and testers can allow these defects to remain undetected.

2. Role of the Tester

Testers should:

- Understand how modules interact.
- Review interface requirements.
- Prepare integration test cases.
- Check data flow between modules.
- Report integration failures clearly.

3. Importance of Early Involvement

Testers should be involved before actual integration begins. This allows them to identify missing requirements, interface problems, and incorrect assumptions early.

4. Example

In a banking system, the **account module** may correctly update the balance, but the **transaction module** may send incorrect transaction information. Integration testing can identify this communication problem.

Conclusion:

Early tester involvement and good developer–tester communication help detect integration defects earlier and improve software reliability.

Q15. Defect Prevention Techniques

Answer:

1. Meaning

Defect prevention focuses on **avoiding defects before they are introduced**, rather than finding them only after development.

2. Techniques

Important techniques include:

- Clear requirement analysis
- Requirement reviews
- Design reviews
- Coding standards
- Code reviews
- Unit testing
- Root cause analysis
- Using historical defect information

3. Example

If the same calculation defect repeatedly occurs in a payroll system, the team can perform root cause analysis and improve the requirement, design, or coding process responsible for the defect.

4. Benefit

Defect prevention:

- Reduces recurring defects.
- Reduces development and testing cost.
- Improves software quality.
- Prevents similar problems in future releases.

Conclusion:

Preventing defects at their source is more effective than repeatedly fixing the same defects after they occur.

Q16. Standardizing Testing Practices for Mobile Applications

Answer:

1. Problem

If a mobile application is tested inconsistently, defects such as crashes may remain undetected.

Different testers may use different devices, test cases, environments, and procedures.

2. Standardization

A standard testing process should define:

- Common test cases
- Supported devices and operating systems
- Testing environments
- Defect reporting format
- Regression testing procedures
- Entry and exit criteria

3. Example

Suppose an application works correctly on one Android phone but crashes on another device. A standardized device compatibility test can help identify this problem.

4. Benefit

Standardized testing:

- Provides consistent coverage.
- Reduces missed defects.
- Makes testing easier to repeat.
- Improves application reliability across devices.

Conclusion:

Standardizing testing practices ensures that important scenarios are tested consistently and reduces the chance of mobile application failures.

Q17. Debugging vs Testing

Answer:

1. Software Testing

Testing is the process of **executing and evaluating software to identify defects or failures**.

2. Debugging

Debugging is the process of **finding the root cause of an identified defect and correcting it**.

3. Difference

Testing	Debugging
Finds defects	Finds and fixes the cause
Mainly performed by testers	Mainly performed by developers
Checks expected vs actual behavior	Examines code and logic
Produces defect reports	Produces code corrections

4. Real-Time Example

In a banking application, a tester notices that ₹5,000 is deducted instead of ₹500.

- **Testing:** identifies and reports the incorrect deduction.
- **Debugging:** developer investigates the transaction calculation and corrects the code.

Conclusion:

Testing identifies problems, while debugging determines their cause and fixes them. Both are important for delivering reliable software.

Q18. Cross-Browser Testing Strategy

Answer:

1. Problem

A shopping website may work correctly in one browser but display incorrect layouts or functionality in another browser.

This can occur because browsers interpret HTML, CSS, and JavaScript differently.

2. Testing Strategy

Testers should:

- Identify supported browsers.
- Test important functions on each browser.
- Check page layout and responsiveness.
- Test login, search, cart, and payment functions.
- Perform regression testing after fixes.

3. Example

Browser	Test
Chrome	Login, search, cart
Firefox	Login, search, cart
Edge	Login, search, cart
Safari	Login, search, cart

4. Benefit

Cross-browser testing ensures that users receive a consistent experience regardless of the supported browser they use.

Conclusion:

A planned cross-browser testing strategy helps identify compatibility defects and improves the reliability and usability of shopping websites.

Q19. Validation Gaps in Logistics Systems

Answer:

1. Problem

If a logistics system produces inconsistent outputs, the problem may be caused by improper input validation, incorrect processing rules, or inconsistent data between modules.

2. Possible Validation Gaps

- Invalid input accepted by the system.
- Missing boundary checks.
- Incorrect data formats.
- Missing mandatory-field validation.
- Different modules applying different validation rules.

3. Example

If a logistics system accepts a negative package weight such as **−10 kg**, it indicates a validation problem. The system should reject the input and display an appropriate error message.

4. Testing Approach

Testers should use:

- Valid inputs
- Invalid inputs
- Boundary values
- Missing values
- Unexpected inputs
- Different combinations of input data

Conclusion:

Proper validation testing prevents inconsistent outputs and improves the reliability of logistics systems.

Q20. Random Testing vs Systematic Test Design

Answer:

1. Problem with Random Testing

Random testing selects inputs without following a structured strategy. Although it may find some unexpected defects, important scenarios can easily be missed.

2. Testing as an Engineering Activity

Software testing should follow a **systematic and planned approach**. Test cases should be designed based on requirements, risks, and suitable testing techniques.

3. Structured Test Design

Testers can use:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Table Testing
- State Transition Testing
- Regression Testing

4. Example

For an age field that accepts values from **18 to 60**, random testing may check 25 and 40 only. Boundary Value Analysis would check values such as **17, 18, 19, 59, 60, and 61**, increasing the chance of finding boundary defects.

Conclusion:

Systematic test design provides better coverage and makes testing more effective than relying only on random test inputs.

Q21. Testing Data Consistency Between Modules

Answer:

1. Problem

In a healthcare system, different modules may store or exchange patient information. If the data is not transferred correctly between modules, inconsistencies can occur.

2. Possible Root Causes

- Incorrect data mapping
- Interface errors
- Database synchronization problems
- Incorrect data formats
- Communication problems between modules

3. Testing Approach

Testers should:

- Enter data in one module.
- Verify whether the same data reaches the connected module.
- Check data format and values.
- Test different valid and invalid inputs.
- Perform integration and regression testing.

4. Example

If a patient's blood group is recorded as **O+** in the registration module but appears as **A+** in the medical records module, it indicates a data consistency defect.

Conclusion:

Integration testing and data validation are important for maintaining consistent and reliable information across healthcare modules.

Q22. Tester Role Across the SDLC

Answer:

1. Importance of Early Involvement

Testers should not be involved only after coding is completed. They should participate throughout the **Software Development Life Cycle (SDLC)**.

2. Tester Activities

- **Requirements:** Identify unclear or missing requirements.
- **Design:** Review design for testability.
- **Development:** Prepare test cases and support unit testing.
- **Integration:** Test interactions between modules.
- **System Testing:** Verify complete system behavior.
- **Maintenance:** Perform regression testing.

3. Example

If a requirement says that a banking transaction should be processed "quickly," the tester can ask for a measurable response-time requirement before development begins.

4. Benefit

Early tester involvement helps identify problems before they become expensive to fix.

Conclusion:

Involving testers throughout the SDLC improves quality, reduces late defects, and supports effective testing.

Q23. Importance of Test Coverage

Answer:

1. Problem

A system may work correctly under normal conditions but fail when many users access it simultaneously. This indicates that important usage scenarios were not sufficiently tested.

2. Test Coverage

Test coverage measures how much of the software, requirements, functions, or scenarios have been tested.

3. Application

For a system expecting high traffic, testers should include:

- Normal usage scenarios
- Peak user load
- Concurrent transactions
- Maximum data volume
- Stress conditions
- Recovery after overload

4. Example

An online ticket-booking system may work with 100 users but fail when 10,000 users access it at the same time. Peak-load scenarios should therefore be included in the test plan.

Conclusion:

Good test coverage ensures that important real-world scenarios are tested and reduces the possibility of failures during peak usage.

Q24. Importance of a Defect Repository

Answer:

1. Meaning

A defect repository is a centralized place where software defects are recorded and tracked throughout their lifecycle.

2. Information Recorded

It may contain:

- Defect ID
- Description
- Severity
- Priority
- Steps to reproduce
- Assigned developer
- Status
- Resolution

3. Application

If defect documentation is missing, the team may forget previous problems. A repository allows testers and developers to track defects from discovery until closure.

4. Example

If a login defect occurs repeatedly, its previous records can be reviewed to identify the cause and create better regression test cases.

Conclusion:

A defect repository improves defect tracking, analysis, communication, and prevention of recurring defects.

Q25. Improving Testing for Banking Transaction Failures

Answer:

1. Problem

Banking transactions require high accuracy. Improper testing can result in incorrect deductions, failed transfers, or incorrect account balances.

2. Improved Testing Approach

The testing process should include:

- Requirement review
- Functional testing
- Boundary Value Analysis
- Integration testing
- Negative testing
- Regression testing
- Security and performance checks

3. Example

For a money-transfer feature, testers should verify:

- Valid transfer amount
- Zero amount
- Negative amount
- Amount greater than account balance
- Daily transaction limit
- Incorrect account number
- Duplicate transaction

4. Benefit

Testing these conditions helps identify transaction failures before deployment.

Conclusion:

A systematic and risk-based testing approach improves the reliability and accuracy of banking transactions.

Q26. Boundary Testing for Missing Edge Cases

Answer:

1. Problem

Missing edge cases can cause software to fail when users provide values close to the allowed limits.

2. Boundary Value Analysis

Boundary Value Analysis focuses on values at and around the boundaries of an input range.

3. Example

Suppose an application accepts an age from **18 to 60**.

Important test values are:

Test Value	Purpose
17	Just below minimum
18	Minimum boundary
19	Just above minimum
59	Just below maximum
60	Maximum boundary
61	Just above maximum

4. Benefit

This approach increases the chance of detecting defects that may not appear with normal values.

Conclusion:

Boundary testing is an effective technique for identifying defects related to input limits and edge cases.

Q27. Process-Based Testing for Consistency

Answer:

1. Problem

Different modules may produce inconsistent results when development and testing processes are not standardized.

2. Process-Based Testing

Process-based testing follows a defined testing process based on requirements, procedures, standards, and expected outputs.

3. Application

The team should:

- Define common testing procedures.
- Use consistent test case formats.
- Follow the same defect reporting process.
- Apply common validation rules.
- Perform regression testing after changes.

4. Example

If two modules calculate customer discounts differently, testers can compare their outputs against the same business rules and identify the inconsistency.

Conclusion:

A consistent testing process helps ensure that different modules behave according to the same requirements and quality standards.

Q28. Testing Unexpected Inputs

Answer:

1. Importance

Software should not be tested only with expected inputs. Unexpected and invalid inputs must also be tested to verify system robustness.

2. Validation Scenarios

Testers can check:

- Empty inputs
- Invalid characters
- Incorrect formats
- Negative values
- Extremely large values
- Missing mandatory fields
- Duplicate inputs

3. Example

For a phone-number field, the tester can enter:

- A valid phone number

- Letters instead of numbers
- An incomplete number
- An empty value
- An excessively long number

The system should reject invalid values and provide a meaningful error message.

4. Benefit

This ensures that unexpected inputs do not cause crashes or incorrect outputs.

Conclusion:

Validation testing improves the robustness, reliability, and user-friendliness of software.

Q29. Defect Leakage to Production

Answer:

1. Meaning

Defect leakage occurs when a defect is not detected during testing and reaches the production environment.

2. Possible Root Causes

- Incomplete test coverage
- Missing test cases
- Insufficient regression testing
- Unclear requirements
- Inadequate test data
- Poor communication between teams

3. Example

Suppose a shopping application works correctly for normal orders but applies an incorrect discount during a special sale. If this scenario was not tested before release, the defect may reach production.

4. Improvement

The team can reduce leakage by:

- Improving test coverage.
- Reviewing requirements.
- Using risk-based testing.
- Maintaining regression suites.
- Analyzing production defects.

Conclusion:

Defect leakage can be reduced by strengthening testing practices and identifying gaps in the development and testing process.

Q30. Redesigning Testing for Early Defect Detection

Answer:

1. Problem

Poor testing strategies often detect defects late in the development lifecycle. Late defect detection increases the cost and effort required for correction.

2. Improved Strategy

The testing process should include:

- Early requirement reviews
- Design reviews
- Tester involvement from the beginning
- Systematic test case design
- Unit and integration testing
- Continuous regression testing
- Defect analysis and prevention

3. Example

If a payroll requirement is ambiguous, identifying the problem during the requirement review is cheaper than discovering it after the payroll system has been developed and deployed.

4. Benefit

Early detection:

- Reduces correction cost.
- Prevents defect propagation.
- Improves software quality.
- Reduces production failures.

Conclusion:

Redesigning the testing strategy to involve testers early and apply systematic testing throughout the SDLC helps detect defects earlier and improves overall software quality.